УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Тестирование программного обеспечения»

Лабораторная работа №4

Вариант 334433

Студенты
Гладкая К. В.
Черкас И. И.
P33141

Преподаватели
Инячина Д. А.
Клименков С. В.

Санкт-Петербург, 2024 г.

Задание

С помощью программного пакета Apache JMeter провести нагрузочное и стресс-тестирование веб-приложения
соответствии с вариантом задания.

В ходе нагрузочного тестирования необходимо протестировать 3 конфигурации аппаратного обеспечения и выбрать среди них наиболее дешёвую, удовлетворяющую требованиям по максимальному времени отклика приложения при заданной нагрузке (в соответствии с вариантом).

В ходе стресс-тестирования необходимо определить, при какой нагрузке выбранная на предыдущем шаге конфигурация перестаёт удовлетворять требованиями по максимальному времени отклика. Для этого необходимо построить график зависимости времени отклика приложения от нагрузки.

Приложение для тестирования доступно только во внутренней сети кафедры.
Если запрос содержит некорректные параметры, сервер возвращает HTTP 403.
Если приложение не справляется с нагрузкой, сервер возвращает HTTP 503.

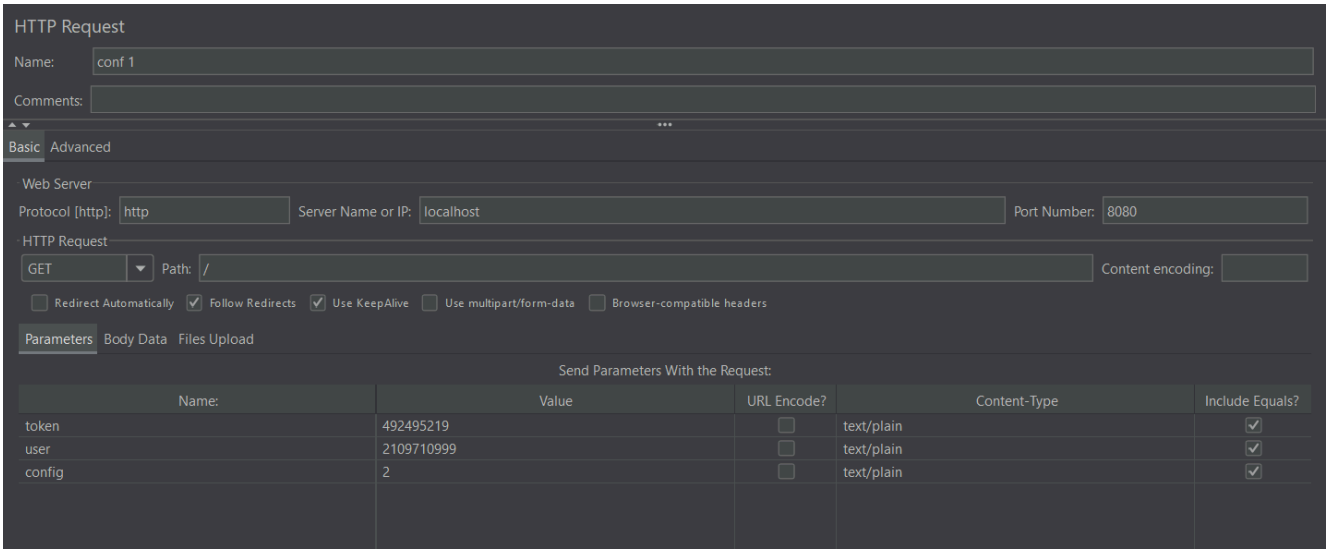
Параметры тестируемого веб-приложения:

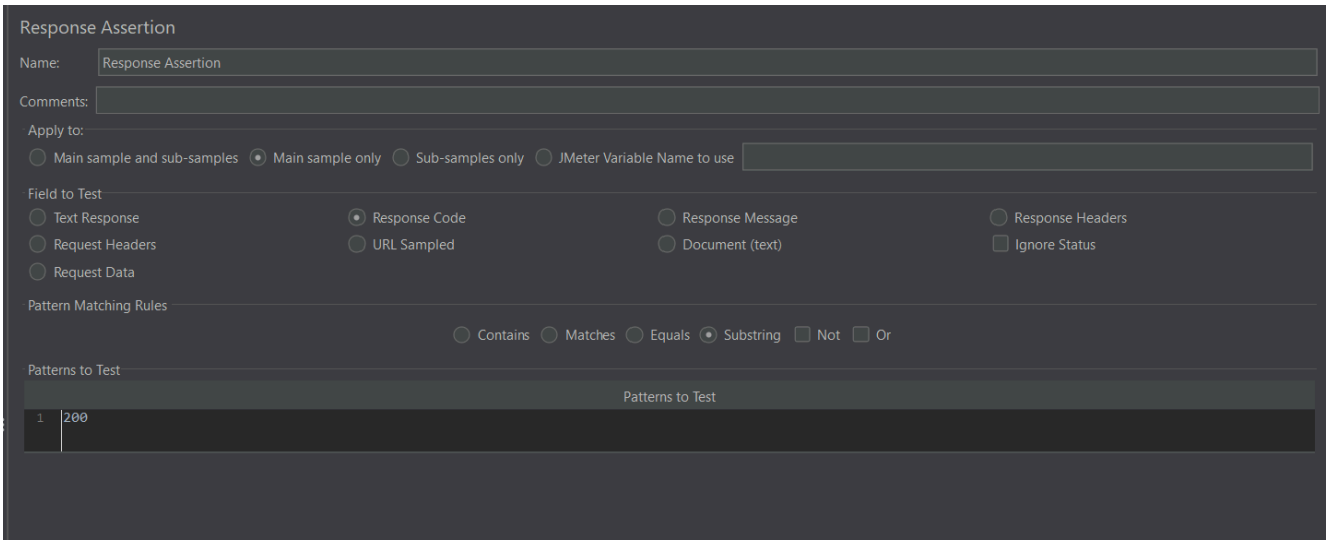
URL первой конфигурации ($ 5200) -
http://stload.se.ifmo.ru:8080?token=492495219&user=2109710999&config=1;
URL второй конфигурации ($ 7200) -
http://stload.se.ifmo.ru:8080?token=492495219&user=2109710999&config=2;
URL третьей конфигурации ($ 10700) -
http://stload.se.ifmo.ru:8080?token=492495219&user=2109710999&config=3;

Максимальное количество параллельных пользователей - 9;
Средняя нагрузка, формируемая одним пользователем - 40 запр. в мин.;
Максимально допустимое время обработки запроса - 790 мс.

Команда для пробрасывания портов:
ssh -v -L 8080:stload.se.ifmo.ru:8080 -p 2222 s334433@helios.se.ifmo.ru
Конфигурации практически совпадают, отличаются только номера параметра config.
Ниже приведен пример конфигурации 2 и её запуска.


Thread Group
Name:
Thread Group
Comments:
Action to be taken after a Sampler error
Continue
Start Next Thread Loop
Stop Thread
Stop Test
Stop Test Now
Thread Properties
Number of Threads (users):
9
Ramp-up period (seconds):
60
Loop Count:
Infinite
20
Same user on each iteration


HTTP Request
Name:
conf 1
Comments:
Basic
Advanced
Web Server
Protocol [http]:
http
Server Name or IP:
localhost
Port Number:
8080
HTTP Request
GET
Path:
/
Content encoding:
Redirect Automatically
Follow Redirects
Use KeepAlive
Use multipart/form-data
Browser-compatible headers
Parameters
Body Data
Files Upload
Send Parameters With the Request:
Name:
Value
URL Encode?
Content-Type
Include Equals?
token
492495219
text/plain
user
2109710999
text/plain
config
2
text/plain


Response Assertion
Name:
Response Assertion
Comments:
Apply to:
Main sample and sub-samples
Main sample only
Sub-samples only
JMeter Variable Name to use
Field to Test
Text Response
Response Code
Response Message
Response Headers
Request Headers
URL Sampled
Document (text)
Ignore Status
Request Data
Pattern Matching Rules
Contains
Matches
Equals
Substring
Not
Or
Patterns to Test
Patterns to Test
1
200

Duration Assertion

Name: Duration Assertion

Comments:

Apply to:

( ) Main sample and sub-samples (•) Main sample only ( ) Sub-samples only

Duration to Assert

Duration in milliseconds: 790

Constant Throughput Timer

Name: Constant Throughput Timer

Comments:

Delay before each affected sampler

Target throughput (in samples per minute): 40.0

Calculate Throughput based on: this thread only

View Results in Table

Name: View Results in Table

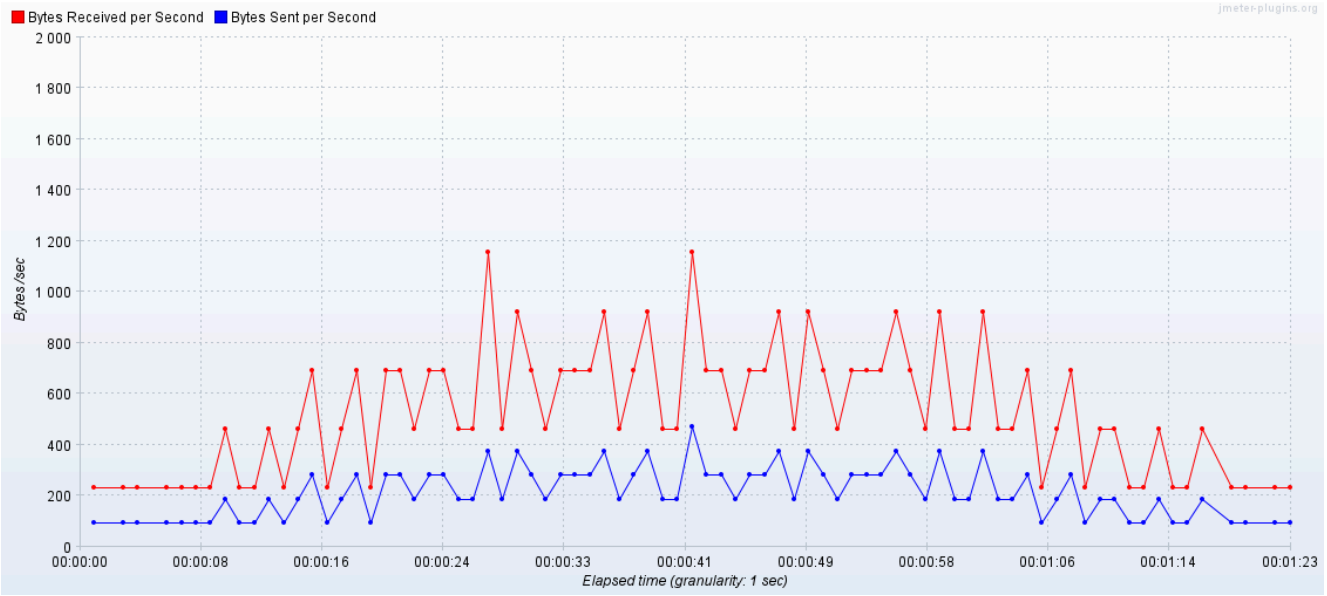
Comments:

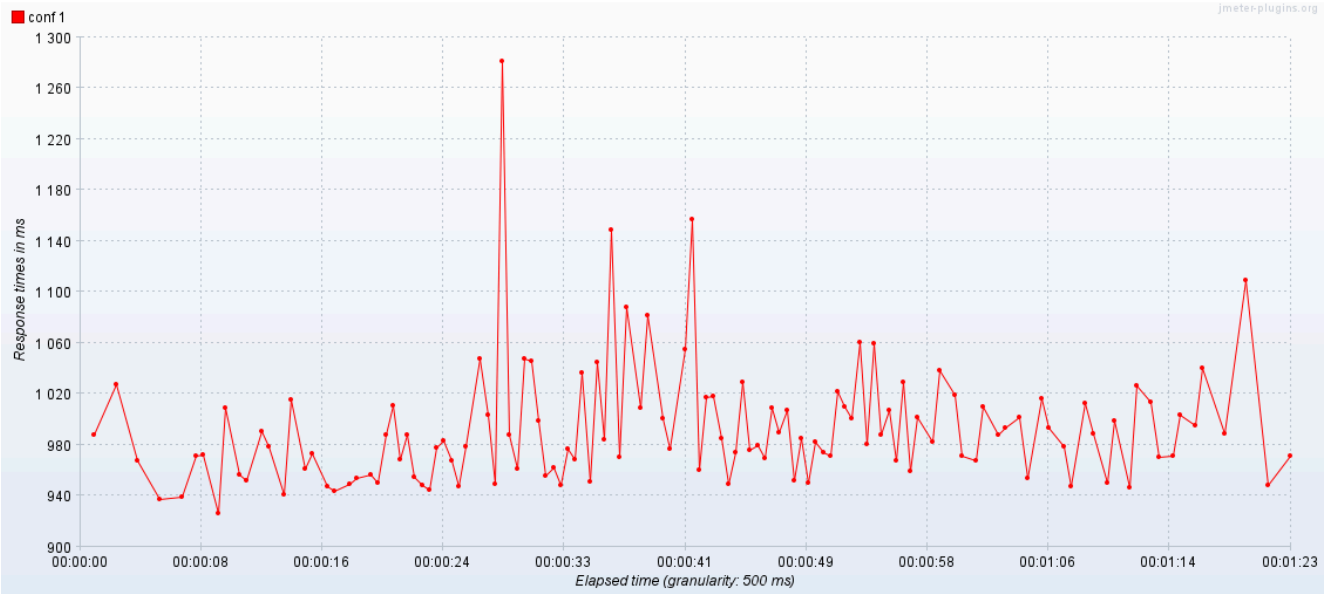
Write results to file / Read from file

Filename Browse... Log/Display Only: Errors Successes Configure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	14:00:23.339	Thread Group 1-1	conf 1	782		231	157	773	17
2	14:00:24.765	Thread Group 1-1	conf 1	731		231	157	731	0
3	14:00:26.264	Thread Group 1-1	conf 1	728		231	157	728	0
4	14:00:27.769	Thread Group 1-1	conf 1	729		231	157	729	0
5	14:00:29.264	Thread Group 1-1	conf 1	725		231	157	725	0
6	14:00:29.921	Thread Group 1-2	conf 1	779		231	157	779	1
7	14:00:30.764	Thread Group 1-1	conf 1	730		231	157	730	0
8	14:00:31.421	Thread Group 1-2	conf 1	734		231	157	734	0
9	14:00:32.264	Thread Group 1-1	conf 1	731		231	157	731	0
10	14:00:32.920	Thread Group 1-2	conf 1	736		231	157	736	0
11	14:00:33.764	Thread Group 1-1	conf 1	735		231	157	735	0
12	14:00:34.420	Thread Group 1-2	conf 1	735		231	157	735	0
13	14:00:35.263	Thread Group 1-1	conf 1	738		231	157	738	0
14	14:00:35.919	Thread Group 1-2	conf 1	735		231	157	734	0
15	14:00:36.586	Thread Group 1-3	conf 1	768		231	157	768	2
16	14:00:36.778	Thread Group 1-1	conf 1	740		231	157	740	0
17	14:00:37.420	Thread Group 1-2	conf 1	737		231	157	737	0
18	14:00:38.087	Thread Group 1-3	conf 1	733		231	157	733	0
19	14:00:38.265	Thread Group 1-1	conf 1	731		231	157	731	0
20	14:00:38.919	Thread Group 1-2	conf 1	736		231	157	736	0
21	14:00:39.586	Thread Group 1-3	conf 1	741		231	157	741	0
22	14:00:39.764	Thread Group 1-1	conf 1	739		231	157	739	0
23	14:00:40.421	Thread Group 1-2	conf 1	732		231	157	732	0
24	14:00:41.088	Thread Group 1-3	conf 1	736		231	157	736	0

Результаты 1-го:
График зависимости отправки данных от времени

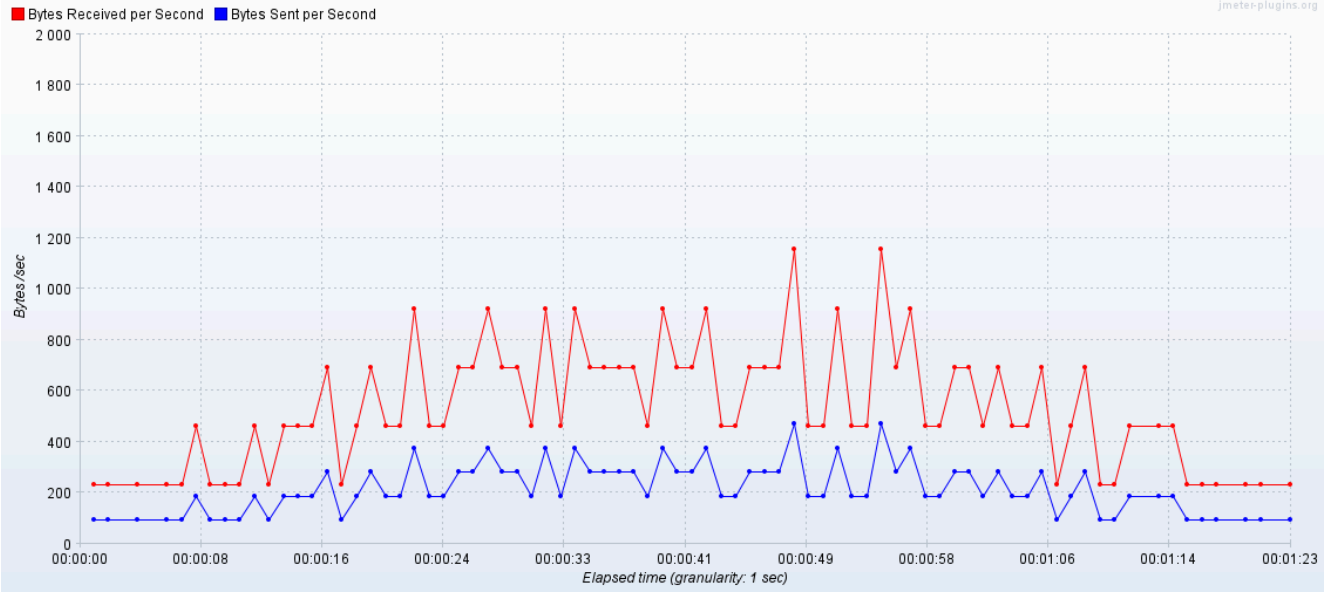


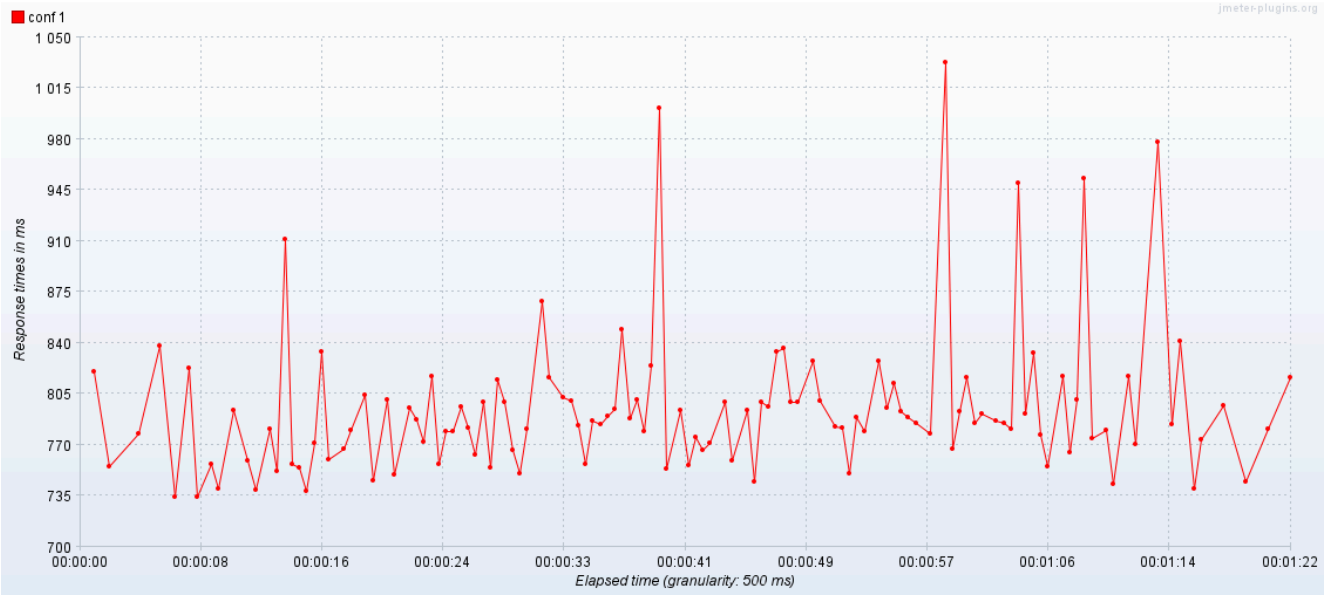


Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
conf 1	180	995	979	1048	1121	1281	926	1507	100,00%	2,2/sec	0,49	0,33
TOTAL	180	995	979	1048	1121	1281	926	1507	100,00%	2,2/sec	0,49	0,33

То есть в 1-ой конфигурации доля ошибок составила 100%

Результаты 2-го:
График зависимости отправки данных от времени

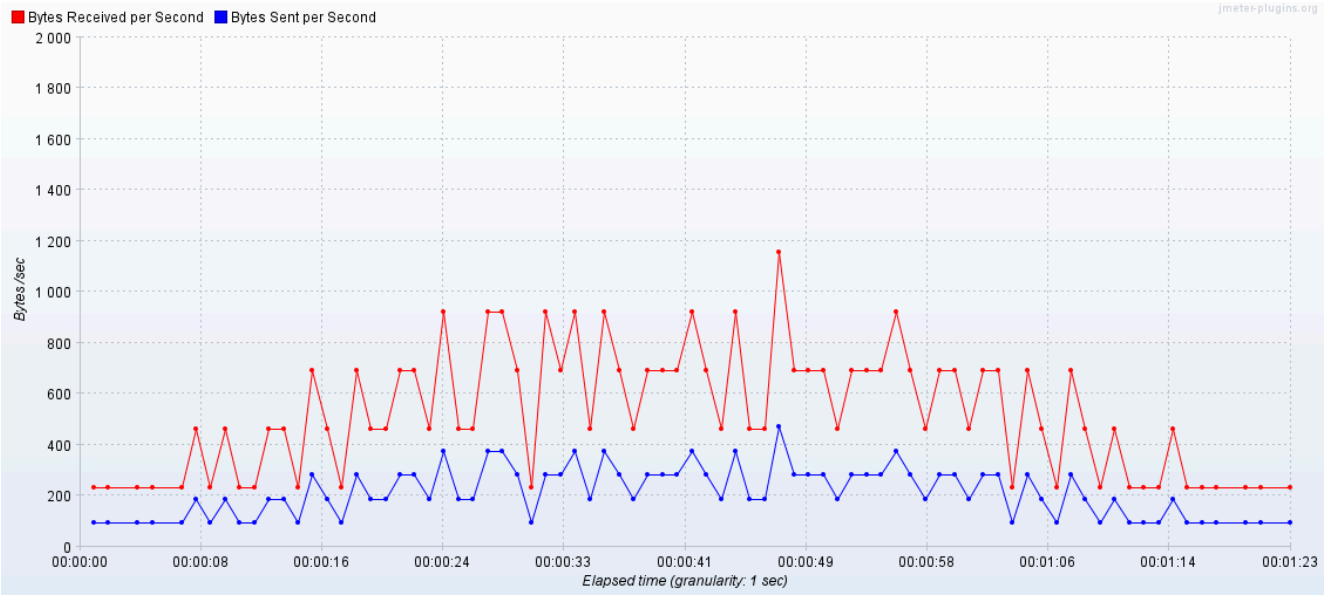


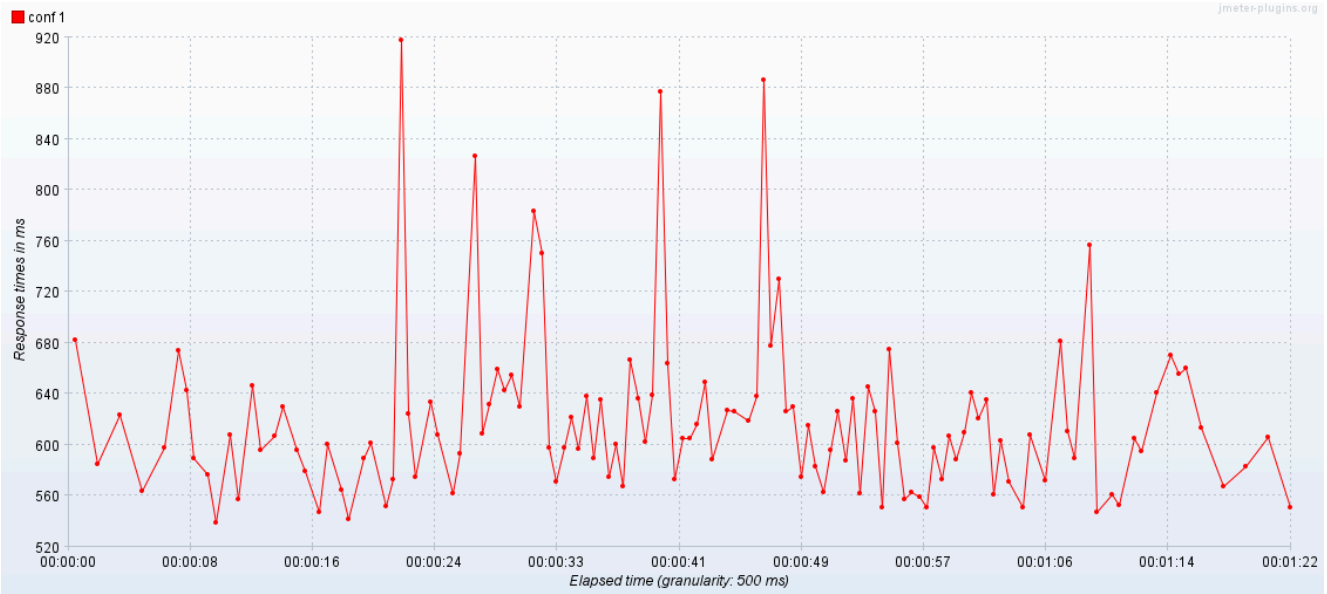


Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
conf 1	180	796	781	841	919	1082	734	1151	42,78%	2,2/sec	0,49	0,33
TOTAL	180	796	781	841	919	1082	734	1151	42,78%	2,2/sec	0,49	0,33

В данной конфигурации процент ошибок уменьшился по сравнению с первой, и стал равен 42,78%

Результаты 3-го:
График зависимости отправки данных от времени



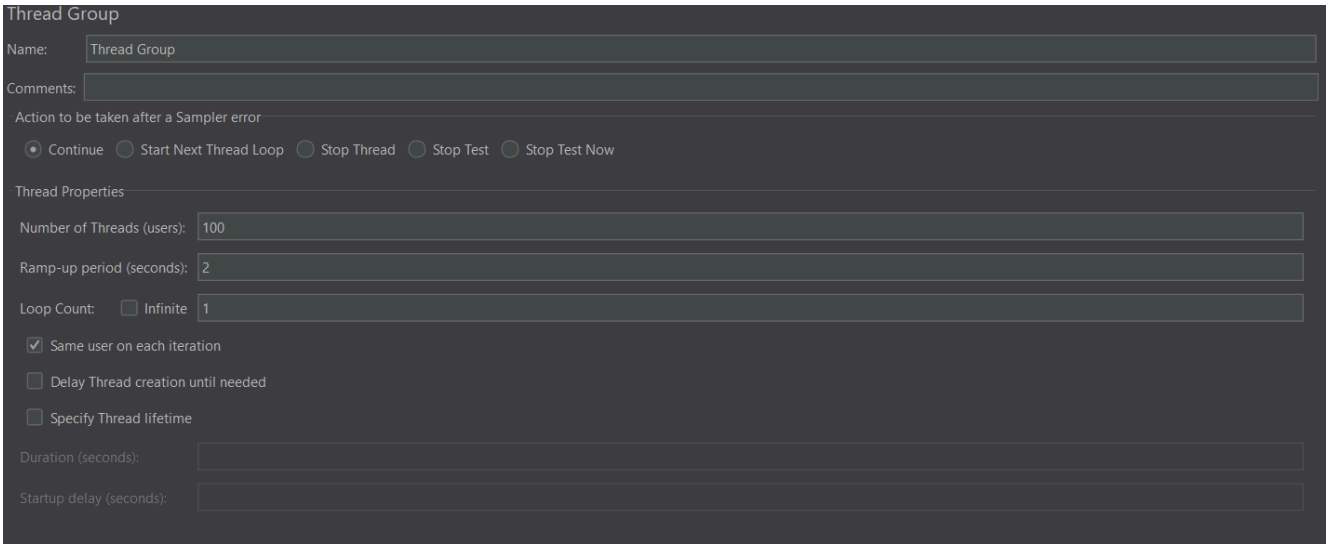


Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
conf 1	180	622	606	686	801	918	539	967	6,11%	2,2/sec	0,49	0,34
TOTAL	180	622	606	686	801	918	539	967	6,11%	2,2/sec	0,49	0,34

Третья конфигурация показала себя лучши всех с результатом в всего 6,11% ошибок

Стресс-тестирования
Мы пришли к выводу, что нет необходимости проводить стресс-тестирования для 1-ой конфигурации, так как там и без этого 100% ошибок

Стресс-тестирование для 2-го:



Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
conf 1	100	972	989	1106	1145	1177	774	1196	98,00%	31,8/sec	7,18	4,88
TOTAL	100	972	989	1106	1145	1177	774	1196	98,00%	31,8/sec	7,18	4,88

Как мы видим по скриншотам, процент ошибок возрос до 98% (то есть увеличился на 55,22%)

Стресс-тестирование для 3-го:



Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
conf 1	100	2174	2150	2527	2540	2559	889	2564	100,00%	38,4/sec	8,66	5,89
TOTAL	100	2174	2150	2527	2540	2559	889	2564	100,00%	38,4/sec	8,66	5,89

Здесь при стресс-тестировании мы пришли к 100% ошибок (то есть процент ошибок увеличился на 93,89)

Вывод

В процессе выполнения лабораторной работы мы познакомились с программой ApacjeJMeter и научились ей пользоваться, а именно настраивать конфигурацию и проводить нагрузочное тестирование. Узнали о разнице конфигураций и посмотрели графики результатов.
А исходя из наших конфигураций, мы пришли к выводу, что при обычном тестировании 3-ая конфигурация показывает наилучший результат, но для стресс-тестирования лучше использовать 2-ую конфигурацию